

RED TEAM LEARNING PATH

MODULE 09

Vulnerability Assessment

Omarchy / Arch Linux Edition

*From component evidence to validated remediation and rescan.***CORE RULE**

Scanner output adalah candidate evidence, bukan otomatis finding final. Presence, applicability, exploitability, exposure, dan business context harus dipisahkan supaya report tidak overclaim.

Version 1.0 | 04 Oktober 2026

1. Tujuan Pembelajaran

- [] Membedakan weakness, vulnerability, CVE, CWE, CPE/package identity, severity, dan risk.
- [] Membaca hasil vulnerability scanner tanpa langsung percaya severity label.
- [] Menggunakan CVSS v4.0 sebagai severity framework, bukan sebagai risk score tunggal.
- [] Menilai product/version match, fixed version, reachability, configuration, dan evidence.
- [] Memahami false positive, false negative, duplicate, accepted risk, dan not-applicable.
- [] Memprioritaskan remediation menggunakan technical severity, exploitation evidence, exposure, dan business criticality.
- [] Melakukan rescan untuk membuktikan remediation, bukan hanya menutup ticket.

2. Prerequisite & Tools

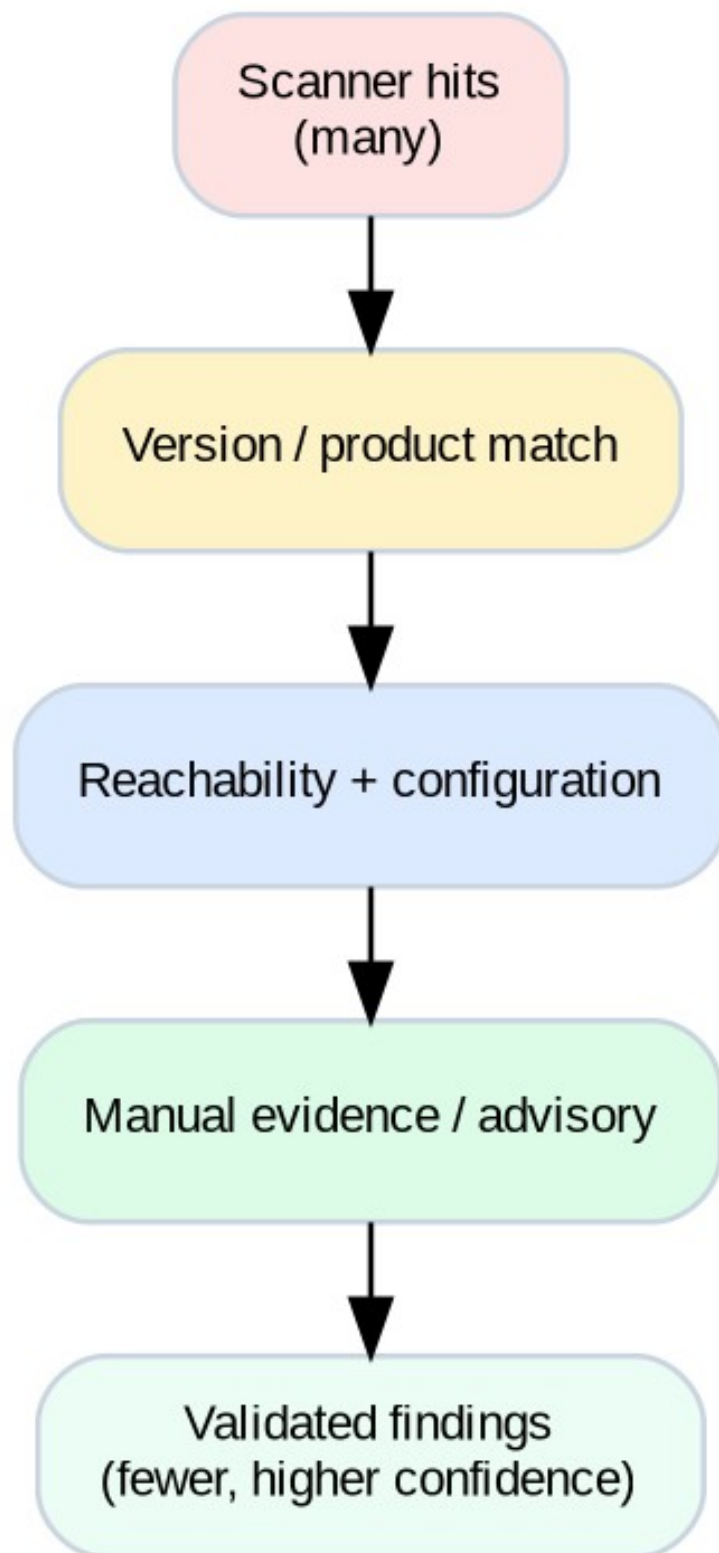
- Module 00-08 sudah dipahami, khususnya scope/RoE, recon, networking, service enumeration, evidence handling, dan scripting dasar.
- Tools utama module: Trivy, jq, Python, optional Syft untuk SBOM, browser untuk CVE/NVD/FIRST/CISA references.
- Lab menggunakan manifest dependency lokal yang sengaja outdated. Tidak ada package vulnerable yang di-install atau dieksekusi.

```
omarchy pkg add trivy syft jq python curl
```

Tool placement

Trivy tersedia di repository resmi Arch dan dapat mendeteksi known vulnerabilities pada OS packages, language packages, container images, dan beberapa software components. Greenbone/OpenVAS dibahas sebagai infrastructure vulnerability management platform, tetapi bukan dependency wajib untuk lab ini.

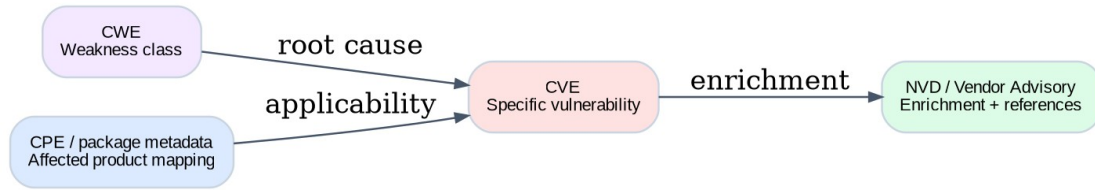
3. Mental Model: Detection -> Validation -> Decision



A large candidate set should shrink as evidence quality increases.

Vulnerability assessment bukan lomba menghasilkan angka findings sebanyak mungkin. Goal-nya adalah membuat daftar keputusan yang bisa dipertanggungjawabkan: apa yang benar-benar affected, seberapa serius, kenapa prioritasnya demikian, dan apa bukti remediation-nya.

4. Terminologi: CWE, CVE, Product Mapping, NVD



How common vulnerability identifiers and enrichment relate conceptually.

Istilah	Makna praktis
CWE	Kelas weakness/root cause, misalnya improper input validation. Bukan instance product tertentu.
CVE	Identifier untuk vulnerability yang publicly disclosed pada product/version tertentu.
CPE / package identity	Cara mengaitkan finding ke product/platform/package yang affected; mapping bisa imperfect.
NVD	NIST database yang meng-enrich CVE dengan metadata seperti CVSS, CWE, references, dan applicability.
Vendor advisory	Sumber penting untuk affected/fixed versions, configuration prerequisites, workaround, dan patch.
KEV	CISA catalog untuk vulnerabilities yang diketahui telah dieksploitasi di dunia nyata.

Weakness != vulnerability

MITRE CWE menjelaskan weakness sebagai kondisi yang dapat berkontribusi pada vulnerability. CVE adalah instance vulnerability pada product tertentu. Jangan menulis report seolah CWE-79 dan CVE adalah jenis identifier yang sama.

5. Candidate Finding vs Validated Finding

Pertanyaan	Evidence yang dicari
Product benar?	Package manager, SBOM, banner + manual verification, vendor inventory.
Version affected?	Vendor advisory / CVE record / distro advisory; perhatikan backport patch.
Component reachable?	Network path, API/function exposure, feature enabled, access control.
Config prerequisite terpenuhi?	Feature flag, plugin/module, auth requirement, deployment mode.
Fix tersedia?	Fixed version, patch, workaround, compensating control.
Eksplorasi aktif?	CISA KEV/vendor threat intel/public exploitation evidence.
Business impact?	Asset criticality, data sensitivity, privilege, blast radius, availability requirements.

Backport trap

Linux distro/vendor dapat backport security patch tanpa mengubah upstream version string seperti yang lo harapkan. Version string saja tidak selalu cukup untuk menyatakan vulnerable. Cek advisory vendor/distro.

6. CVSS v4.0: Severity, Bukan Risk Tunggal

CVSS v4.0 adalah framework FIRST untuk mengkomunikasikan technical characteristics dan severity. Metric groups utama: Base, Threat, Environmental, dan Supplemental. Base score 0-10 berguna sebagai baseline, tetapi prioritas remediation seharusnya mempertimbangkan Threat dan Environmental context bila tersedia.

Metric group	Pertanyaan
Base	Seberapa berat intrinsic technical characteristics vulnerability?
Threat	Apa yang berubah berdasarkan exploit maturity/threat saat ini?
Environmental	Bagaimana karakteristik environment kita mengubah dampak/importance?
Supplemental	Informasi tambahan seperti safety, automatable, recovery, provider urgency; tidak mengubah score final.

NVD reminder

NVD secara eksplisit menyatakan CVSS adalah ukuran severity, bukan ukuran risk. Jangan menerjemahkan CVSS 9.8 menjadi "risk 9.8" tanpa business/exposure context.

Saat mencatat CVSS, simpan score DAN vector. Vector membuat orang lain dapat memahami metric choices, bukan hanya melihat angka akhir.

```
Example notation:
CVSS-B: 8.7
Vector: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/...
```

Gunakan FIRST calculator untuk scoring; jangan menebak vector dari label scanner.

7. Scanner Mechanics & Limitations

- Signature/database scanner bekerja dari evidence seperti package name/version, CPE, advisory metadata, configuration checks, atau network probes.
- Database freshness mempengaruhi hasil. Scanner hari ini dan bulan depan dapat memberi result berbeda pada input yang sama.
- Unauthenticated network scan hanya melihat apa yang exposed dari luar; credentialed/local scan biasanya punya visibility jauh lebih tinggi.
- Scanner dapat salah mapping package/product, gagal mengenali vendor backport, atau tidak memahami apakah vulnerable code path benar-benar reachable.
- False negative juga mungkin: komponen custom, third-party binary, runtime-only dependency, shadow IT, atau database coverage yang tidak lengkap.

8. Scanner Types

Jenis	Contoh	Kekuatan	Keterbatasan
SCA / filesystem	Trivy, Dependency-Check	Dependency/component visibility	Presence tidak sama dengan exploitability.
Container / image	Trivy	OS + language packages sebelum deploy	Image != runtime exposure.
Infrastructure VM	Greenbone/OpenVAS	Network services + vulnerability tests	Butuh tuning, credentials, dan impact awareness.
Authenticated host	Scanner agent/credentialed scan	Package/config visibility tinggi	Credential handling dan privilege harus aman.
DAST/web	Burp/ZAP class tools	Runtime behavior	Coverage bergantung crawl/auth/business flows.

9. Prioritization: Severity + Threat + Exposure + Business

Priority yang berguna menjawab "apa yang harus diperbaiki dulu?", bukan sekadar mengurutkan CVSS. Gunakan technical severity sebagai salah satu input, lalu tambahkan exploitation evidence, exposure, asset role, privilege path, data sensitivity, dan compensating controls.

Signal	Contoh interpretasi
Known exploited	CISA KEV atau vendor mengonfirmasi active exploitation -> urgency naik.
Internet exposed	Attack surface terbuka tanpa internal foothold -> urgency naik.
Privileged service	Compromise memberi SYSTEM/root/domain impact -> urgency naik.
Business critical	Downtime/data loss berdampak tinggi -> impact context naik.
Strong compensating control	Segmentation, feature disabled, WAF/ACL, no vulnerable path -> residual risk bisa turun.
No fix yet	Gunakan vendor workaround/mitigation dan monitoring; jangan sekadar "accept" tanpa owner/expiry.

KEV is a priority signal

CISA KEV sengaja dibuat untuk membantu organisasi memprioritaskan vulnerabilities yang sudah diketahui dieksploitasi. Tetap validasi apakah product/version lo affected dan apakah asset berada dalam environment yang relevan.

10. Remediation Quality

1. Tentukan root cause/component yang harus diubah.
2. Pilih patch/upgrade/vendor workaround yang supported.
3. Uji compatibility di environment yang sesuai.
4. Deploy dengan change control.
5. Rescan atau re-test menggunakan evidence path yang sama.
6. Bandingkan before/after dan simpan bukti.
7. Tutup finding hanya jika acceptance criteria terpenuhi.

Fix != silence scanner

Menyembunyikan finding via ignore/suppression bukan remediation. Suppression hanya legitimate bila ada documented justification seperti false positive, not-applicable, accepted risk dengan owner/expiry, atau VEX/vendor statement yang valid.

11. Lab Design



Module 09 lab scans a local manifest only; vulnerable dependencies are never executed.

Fixture berisi beberapa dependency version lama untuk menghasilkan candidate findings. Exact CVE count dapat berubah karena database vulnerability terus diperbarui. Itu justru bagian dari pelajaran: evidence harus punya timestamp dan database context.

12. Hands-on Lab 1 - Workspace & Training Marker

```
./labs/va-init.sh module09-va
cat module09-va/00-scope/README.md
```

run-trivy.sh hanya mau scan path yang punya marker .module09-training-target. Guardrail ini sengaja mencegah script latihan dipakai untuk scan filesystem arbitrary secara tidak sengaja.

13. Hands-on Lab 2 - Create Intentionally Outdated Manifest

```
./labs/make-fixture.sh module09-va
cat module09-va/01-target/requirements.txt
```

Safe fixture

Dependency tidak di-install dan tidak dieksekusi. Kita hanya memberi scanner sebuah manifest untuk dipetakan terhadap known vulnerability data.

14. Hands-on Lab 3 - First Trivy Scan

```
./labs/run-trivy.sh module09-va
```

```
jq '.Results[] | {Target,Type,count:(.Vulnerabilities // [] | length)}' module09-va/02-scans/trivy.json
```

First run Trivy biasanya perlu mengambil vulnerability database. Simpan waktu scan dan tool version karena output dapat berubah seiring database update.

```
trivy --version
date -u +%FT%TZ
```

15. Hands-on Lab 4 - Normalize Evidence

```
python labs/normalize-trivy.py \
  module09-va/02-scans/trivy.json \
  --output module09-va/03-normalized/findings.csv
```

```
column -s, -t < module09-va/03-normalized/findings.csv | less -S
```

JSON tetap menjadi raw evidence. CSV hanya view terstruktur untuk triage. Jangan menghapus sumber mentah setelah normalisasi.

16. Hands-on Lab 5 - Manual CVE Validation

Pilih satu CVE dari CSV. Untuk CVE tersebut, jawab secara tertulis:

- [] Apa product/package yang affected?
- [] Version range affected apa?
- [] Apa fixed version menurut scanner/vendor advisory?
- [] CWE/root cause apa yang terkait?
- [] Apa prerequisites/attack vector?
- [] Apakah hanya presence yang terbukti, atau code path/reachability juga terbukti?
- [] Apakah ada vendor-specific distro advisory yang mengubah applicability?

Gunakan CVE Record/vendor advisory sebagai identitas vulnerability, NVD sebagai enrichment/reference, dan CWE untuk weakness class. Jangan mengandalkan satu blog/random exploit repository sebagai source of truth.

17. Hands-on Lab 6 - Read CVSS v4, Jangan Copy Blindly

1. Buka FIRST CVSS v4 calculator.
2. Ambil CVSS vector dari authoritative advisory jika tersedia.
3. Baca setiap metric dan tulis kenapa nilainya masuk akal.
4. Bandingkan Base dengan Threat/Environmental context di lab.
5. Catat score + vector, bukan score saja.

Exercise

Ambil satu finding severity HIGH dari scanner. Bayangkan asset hanya offline build artifact dan vulnerable dependency tidak pernah dieksekusi. Technical severity CVE tidak berubah, tetapi remediation priority environment dapat berubah. Jelaskan perbedaannya.

18. Hands-on Lab 7 - Triage Decision Log

```
python labs/finding-note.py \
  module09-va/04-triage/decisions.csv \
  --cve CVE-YYYY-NNNN \
  --package urllib3 \
  --decision needs-more-evidence \
  --reason "package present; runtime reachability not established" \
  --priority P3
```

Decision log memaksa lo memisahkan "scanner says vulnerable" dari "gue sudah punya evidence untuk claim ini".

19. Hands-on Lab 8 - Optional SBOM View

```
syft dir:module09-va/01-target \
  -o cyclonedx-json=module09-va/06-evidence/sbom.cdx.json
```

SBOM membantu menjawab komponen apa yang terdeteksi. SBOM bukan vulnerability report dan tidak membuktikan exploitability. Ia menjadi input inventory yang dapat dikorelasikan dengan vulnerability intelligence.

20. Hands-on Lab 9 - Remediate One Dependency

Dari finding yang punya FixedVersion, pilih satu package. Jangan copy version dari modul ini karena fixed version dapat berubah; gunakan current authoritative scanner/vendor data.

```
python labs/remediate-one.py \
  module09-va/01-target/requirements.txt \
  --package urllib3 \
  --version <FIXED_VERSION_FROM_CURRENT_EVIDENCE>
```

Script membuat backup requirements.txt.bak. Review diff sebelum rescans:

```
diff -u module09-va/01-target/requirements.txt.bak \
  module09-va/01-target/requirements.txt || true
```

21. Hands-on Lab 10 - Rescan & Compare

```
cp module09-va/02-scans/trivy.json \
  module09-va/05-remediation/trivy-before.json

./labs/run-trivy.sh module09-va

cp module09-va/02-scans/trivy.json \
  module09-va/05-remediation/trivy-after.json
```

Normalize before/after secara terpisah, lalu cek apakah CVE yang ditargetkan hilang/berubah. Finding lain boleh tetap ada - goal lab adalah membuktikan satu remediation dengan evidence repeatable.

22. Evidence Integrity

```
./labs/evidence-manifest.sh module09-va
cat module09-va/06-evidence/sha256-manifest.txt
```

Integrity scope

SHA-256 membuktikan file tidak berubah sejak manifest dibuat. Hash tidak membuktikan scanner benar, target benar, atau interpretation benar.

23. Mini Project - Vulnerability Case File

```
module09-va/
├── 00-scope/
├── 01-target/requirements.txt
├── 02-scans/trivy.json
├── 03-normalized/findings.csv
├── 04-triage/decisions.csv
├── 05-remediation/
├── 06-evidence/
└── 07-summary/vulnerability-summary.md
```

Summary minimal: target/component, tool+DB timestamp, candidate finding, manual validation status, severity/vector, exposure/business assumptions, remediation, rescan result, dan remaining uncertainty.

24. Practical Challenge - 45 Menit

- [] Buat workspace dan fixture baru.
- [] Jalankan Trivy dan simpan raw JSON.
- [] Normalize ke CSV.
- [] Pilih minimal dua CVE dari package berbeda.
- [] Untuk masing-masing: cek authoritative CVE/advisory, affected range, fixed version, CWE bila ada, dan CVSS vector.
- [] Tulis satu finding validated/likely dan satu finding needs-more-evidence/not-applicable dengan alasan.
- [] Pilih satu dependency dengan fixed version dan lakukan remediation manifest.
- [] Rescan dan buktikan before/after.
- [] Buat SHA-256 evidence manifest.
- [] Tulis summary yang tidak melebihi-lebihkan exploitability.

Challenge rule

Lo tidak perlu menjalankan exploit untuk memvalidasi vulnerability assessment dasar. Untuk banyak finding, product/version/configuration/advisory evidence sudah cukup. Exploitation hanya dilakukan nanti jika scope, safety, dan objective engagement memang membutuhkan proof yang lebih dalam.

25. Common Mistakes

Kesalahan	Impact	Perbaikan
CVSS = risk	Prioritas salah	Gabungkan severity dengan threat/exposure/business context.
Version match = vulnerable	False positive	Cek vendor advisory, backport, config, reachability.
Ignore fixed version	Finding tidak actionable	Selalu capture remediation source/fixed version.
Scan DB stale	Miss/incorrect findings	Catat DB/tool timestamp dan update sesuai policy.
Unauth scan dianggap lengkap	False sense of coverage	Pahami visibility dan gunakan authenticated/local evidence bila authorized.
Menjalankan exploit untuk semua CVE	Operational risk	Gunakan least-invasive validation evidence lebih dulu.
Close ticket setelah patch deploy	Belum terbukti	Rescan/retest dan simpan before/after evidence.
Suppress tanpa rationale	Risk tersembunyi	Owner, reason, expiry, compensating controls harus jelas.

26. Knowledge Check

1. Apa beda CWE dan CVE?
2. Kenapa NVD mengatakan CVSS bukan risk score?
3. Apa bedanya presence, applicability, reachability, dan exploitability?
4. Kenapa vendor backport dapat membuat version-only matching salah?
5. Apa beda authenticated dan unauthenticated vulnerability scan?
6. Kapan finding bisa dianggap not-applicable?
7. Kenapa KEV mempengaruhi prioritization?
8. Apa fungsi fixed version dalam remediation workflow?
9. Kenapa raw JSON scanner tetap disimpan setelah dibuat CSV?
10. Apa bukti minimum untuk menutup finding setelah patch?

27. Exit Criteria

- [] Saya bisa membedakan CWE, CVE, CVSS, NVD, vendor advisory, dan KEV.
- [] Saya tidak menyebut scanner hit sebagai confirmed vulnerability tanpa validation.
- [] Saya bisa membaca Trivy JSON dan membuat normalized inventory.
- [] Saya bisa mencatat score + CVSS vector dan menjelaskan kenapa severity bukan risk tunggal.
- [] Saya bisa membuat triage decision dengan evidence/rationale.
- [] Saya bisa memperbaiki satu dependency berdasarkan current fixed-version evidence lalu rescans.
- [] Saya bisa menjelaskan false positive dan false negative secara realistis.
- [] Saya memahami kapan exploitation tidak diperlukan untuk vulnerability assessment.

28. Cheat Sheet

```
# Install tools
omarchy pkg add trivy syft jq python curl

# Workspace + safe fixture
./labs/va-init.sh module09-va
./labs/make-fixture.sh module09-va

# Scan
./labs/run-trivy.sh module09-va
jq '.Results[] | {Target,Type,count:(.Vulnerabilities // [] | length)}' \
  module09-va/02-scans/trivy.json

# Normalize
python labs/normalize-trivy.py module09-va/02-scans/trivy.json \
  --output module09-va/03-normalized/findings.csv

# Manual triage log
python labs/finding-note.py module09-va/04-triage/decisions.csv \
  --cve CVE-YYYY-NNNN --package PACKAGE \
  --decision needs-more-evidence --reason "..." --priority P3

# Evidence integrity
./labs/evidence-manifest.sh module09-va
```

29. Referensi Utama

NIST NVD - Vulnerability Detail Pages - NVD enrichment, references, dan reminder bahwa CVSS mengukur severity, bukan risk.

<https://nvd.nist.gov/vuln/Vulnerability-Detail-Pages>

FIRST - CVSS v4.0 User Guide - Base, Threat, Environmental, Supplemental metrics dan scoring guidance.

<https://www.first.org/cvss/v4.0/user-guide>

MITRE CWE - FAQ - Definisi weakness serta perbedaan weakness vs vulnerability.

<https://cwe.mitre.org/about/faq>

CVE Program - Glossary - CVE ID, CVE Record, CNA, dan terminology resmi CVE Program.

<https://www.cve.org/ResourcesSupport/Glossary>

CISA - Known Exploited Vulnerabilities Catalog - Prioritization signal untuk vulnerabilities yang known exploited.

<https://www.cisa.gov/known-exploited-vulnerabilities-catalog>

Trivy - Vulnerability Scanning - Coverage dan data sources Trivy.

<https://trivy.dev/docs/latest/scanner/vulnerability/>

Arch Linux - trivy package - Trivy tersedia di repository resmi Arch.

https://archlinux.org/packages/extra/x86_64/trivy/

OWASP Dependency-Check - SCA alternative untuk known vulnerable dependencies.

<https://owasp.org/projects/dependency-check>

Greenbone Community Documentation - OpenVAS/Greenbone vulnerability management stack.

<https://greenbone.github.io/docs/latest/>

NIST SP 800-115 - Benefits, limitations, dan workflow technical security assessment.

<https://csrc.nist.gov/pubs/sp/800/115/final>

Next module

Module 10 - Burp Suite Fundamentals: proxy, HTTP history, Repeater, scope, TLS interception, request editing, evidence discipline, dan local web lab workflow.